



ENGSE207 Software Architecture

Lab2: Assignment Architectural Drivers

เสนอ

อาจารย์ธนิต เกตุแก้ว

จัดทำโดย

67543210031-0 นายธนภัทร นุกูล

67543210072-4 นายวรวัฒน์ จันท์เที่ยง

67543210074-0 นางสาวสุพิชญา ชื่นจุม

คณะวิศวกรรมศาสตร์

สาขาวิชา วศ.บ.วิศวกรรมซอฟต์แวร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา เชียงใหม่

## ส่วนที่ 1: System Overview

### 1. System Description

LINE MAN คือระบบส่งอาหารออนไลน์ (Food Delivery System) ที่เชื่อมโยงผู้บริโภค ร้านอาหาร และพนักงานขับรถส่งอาหาร เข้าด้วยกันผ่านแพลตฟอร์มเดียว ระบบช่วยให้ผู้ใช้สามารถค้นหาร้านอาหาร เลือกเมนู สั่งอาหาร และชำระเงินได้อย่างสะดวกรวดเร็วผ่านแอปพลิเคชันบนสมาร์ทโฟน ระบบมีการทำงานแบบ Real-time ตั้งแต่การรับออเดอร์ การแจ้งเตือนร้านอาหาร การจัดส่ง ไปจนถึงการติดตามตำแหน่งของพนักงานส่งอาหารบนแผนที่ ร้านอาหารสามารถจัดการเมนู ราคา และสถานะออเดอร์ผ่านแดชบอร์ดของร้านค้า ขณะที่ผู้ดูแลระบบสามารถบริหารจัดการผู้ใช้ ร้านค้า และข้อมูลสถิติทั้งหมด ปัญหาหลักที่ระบบนี้แก้ไข ได้แก่ ความไม่สะดวกในการโทรสั่งอาหารแบบเดิม ความไม่แน่นอนเรื่องเวลาจัดส่ง และการขาดความโปร่งใสในกระบวนการสั่งและรับอาหาร LINE MAN ช่วยลด Friction ในทุกขั้นตอนและสร้าง Ecosystem ที่เชื่อมโยงกันแบบ End-to-End

### 2. Target Users

- ลูกค้า (Customer): บุคคลทั่วไปที่ต้องการสั่งอาหารออนไลน์ผ่านแอปพลิเคชัน มีอายุ 15-50 ปี ใช้งานสมาร์ทโฟน และต้องการความสะดวกรวดเร็ว
- ร้านอาหาร (Restaurant Partner): ร้านอาหารที่ลงทะเบียนเป็นพาร์ทเนอร์ ต้องการช่องทางการขายเพิ่มเติมและระบบจัดการออเดอร์ที่มีประสิทธิภาพ
- พนักงานส่งอาหาร (Rider): บุคคลที่รับงานส่งอาหารผ่านแอปฯ Rider สามารถเลือกรับงานได้อิสระ และต้องการระบบนำทางและแจ้งเตือนที่แม่นยำ
- ผู้ดูแลระบบ (Admin): ทีม Ops ของบริษัทที่ดูแลการทำงานของแพลตฟอร์ม จัดการผู้ใช้ ร้านค้า และแก้ไขปัญหาต่างๆ

### 3. Key Features (Top 10)

- ค้นหาและเรียกดูร้านอาหารตามพื้นที่และหมวดหมู่
- เลือกเมนูและเพิ่มลงตะกร้า พร้อมปรับแต่งรายการพิเศษ
- ชำระเงินออนไลน์ผ่าน QR Code, บัตรเครดิต และ Wallet
- ติดตามสถานะออเดอร์แบบ Real-time บนแผนที่
- ระบบแจ้งเตือน Push Notification ทุกขั้นตอนของออเดอร์
- จัดการเมนูและราคาสำหรับร้านอาหาร (Restaurant Dashboard)
- ระบบรับและจัดการงานส่งของสำหรับ Rider (Rider App)
- รีวิวและให้คะแนนร้านอาหารและ Rider หลังได้รับอาหาร
- โปรโมชั่น ส่วนลด และ Voucher สำหรับลูกค้า
- รายงานสถิติและยอดขายสำหรับร้านอาหารและ Admin

## ส่วนที่ 2: Functional Requirements

### User Management

- FR-01: ลูกค้าสามารถลงทะเบียนบัญชีผ่าน Email หรือ Phone Number ได้
- FR-02: ผู้ใช้ทุกประเภทสามารถ Login/Logout พร้อมรองรับ 2-Factor Authentication
- FR-03: ผู้ใช้สามารถแก้ไขข้อมูลส่วนตัว เช่น ชื่อ เบอร์โทร และรูปโปรไฟล์
- FR-04: Admin สามารถระงับ/ยกเลิกบัญชีผู้ใช้ที่ผิดเงื่อนไขได้

### Core Features (Order)

- FR-05: ลูกค้าสามารถค้นหาร้านอาหารตามพิกัด GPS และกรองตามหมวดหมู่/ระยะทาง
- FR-06: ลูกค้าสามารถเพิ่มสินค้าลงตะกร้าและแก้ไขก่อนสั่งซื้อ
- FR-07: ระบบสามารถคำนวณค่าส่งตามระยะทางแบบ Dynamic ได้
- FR-08: ลูกค้าสามารถชำระเงินผ่าน QR Code, บัตรเครดิต/เดบิต และ LINE Pay
- FR-09: ระบบจะส่ง Order ไปยังร้านอาหารทันทีหลังชำระเงินสำเร็จ
- FR-10: ร้านอาหารสามารถยืนยัน/ปฏิเสธ Order พร้อมระบุเหตุผลได้
- FR-11: Rider สามารถรับงาน Pickup และ Deliver Order ผ่านแอปฯ ได้

## Tracking & Notification

- FR-12: ระบบแสดงตำแหน่ง Rider บนแผนที่แบบ Real-time ให้ลูกค้าเห็น
- FR-13: ระบบส่ง Push Notification เมื่อสถานะ Order เปลี่ยน เช่น "ร้านกำลังเตรียมอาหาร", "Rider รับออเดอร์แล้ว"
- FR-14: ลูกค้าสามารถยกเลิก Order ได้ภายในเวลาที่กำหนดก่อนร้านยืนยัน

## Review & Rating

- FR-15: ลูกค้าสามารถให้คะแนน (1-5 ดาว) และเขียนรีวิวร้านอาหาร/Rider หลังรับอาหาร

## Reporting

- FR-16: ร้านอาหารสามารถดูรายงานยอดขายรายวัน/สัปดาห์/เดือนใน Dashboard
- FR-17: Admin สามารถ Export รายงาน Transaction และสถิติการใช้งานระบบเป็นไฟล์ CSV/PDF

## ส่วนที่ 3: Quality Attributes & Scenarios

### QA-1: Performance (ประสิทธิภาพ)

| ส่วนประกอบ       | รายละเอียด                                                                   |
|------------------|------------------------------------------------------------------------------|
| Source           | ลูกค้าจำนวนมากพร้อมกันช่วง Lunch Peak (12:00-13:00)                          |
| Stimulus         | ลูกค้า 5,000 คนกดยืนยัน Order พร้อมกันในช่วง Peak Hour                       |
| Artifact         | Order Processing Service และ Payment Gateway                                 |
| Environment      | ระบบทำงาน Production ในช่วง Peak Load                                        |
| Response         | ระบบรับและประมวลผล Order ทุกรายการโดยไม่มีข้อมูลสูญหาย                       |
| Response Measure | Response Time $\leq$ 2 วินาที สำหรับ 95th Percentile และ Error Rate $<$ 0.1% |

คำอธิบาย: ช่วง Peak Hour เป็นช่วงเวลาวิกฤตของระบบ หาก Response Time สูงเกินไปหรือมี Error จะส่งผลให้ลูกค้าเสียประสบการณ์และอาจออกจากแอปฯ การรับประกันว่าระบบสามารถรองรับ Load ในช่วงนี้ได้จึงสำคัญมาก

### QA-2: Scalability (ความสามารถในการขยาย)

| ส่วนประกอบ       | รายละเอียด                                                                     |
|------------------|--------------------------------------------------------------------------------|
| Source           | บริษัทต้องการขยายบริการสู่จังหวัดใหม่ 5 จังหวัด                                |
| Stimulus         | จำนวน Active Users เพิ่มขึ้น 3 เท่าภายใน 1 เดือน (จาก 100,000 เป็น 300,000 คน) |
| Artifact         | Infrastructure ทั้งระบบ (Application Server, Database, Cache)                  |
| Environment      | ระบบกำลังทำงาน Production โดยไม่มี Downtime                                    |
| Response         | ระบบปรับขนาด (Auto-scale) โดยอัตโนมัติรองรับ Traffic ที่เพิ่มขึ้น              |
| Response Measure | ขยาย Capacity ได้ภายใน 5 นาที และ Availability $\geq$ 99.5% ตลอดช่วงการขยาย    |

คำอธิบาย: การขยายตลาดต้องการให้ระบบรองรับผู้ใช้ใหม่ได้อย่างราบรื่น โดยไม่กระทบประสบการณ์ผู้ใช้เดิม การออกแบบระบบให้รองรับ Horizontal Scaling ด้วย Cloud Infrastructure เป็นสิ่งจำเป็น

QA-3: Availability (ความพร้อมใช้งาน)

| ส่วนประกอบ       | รายละเอียด                                                                               |
|------------------|------------------------------------------------------------------------------------------|
| Source           | Hardware Failure หรือ Server Crash โดยไม่คาดคิด                                          |
| Stimulus         | Database Primary Node หยุดทำงานกะทันหันในเวลากลางคืน                                     |
| Artifact         | Database Cluster และ Order Management Service                                            |
| Environment      | ระบบทำงาน Production นอกช่วง Maintenance Window                                          |
| Response         | ระบบ Failover ไปยัง Replica Node โดยอัตโนมัติ และ Order ที่อยู่ระหว่างดำเนินการไม่สูญหาย |
| Response Measure | Downtime < 30 วินาที, Recovery Point Objective (RPO) = 0, Availability ≥ 99.9% ต่อปี     |

คำอธิบาย: ระบบ Food Delivery ทำงานตลอด 24 ชั่วโมง การที่ระบบล่มแม้เพียงไม่กี่นาทีส่งผลเสียต่อรายได้ทั้งร้านอาหาร Rider และลูกค้า การออกแบบ High Availability ด้วย Active-Passive Failover จึงเป็นสิ่งจำเป็น

QA-4: Security (ความปลอดภัย)

| ส่วนประกอบ       | รายละเอียด                                                                                  |
|------------------|---------------------------------------------------------------------------------------------|
| Source           | ผู้ไม่หวังดี (Attacker) จากภายนอก                                                           |
| Stimulus         | พยายาม Brute Force รหัสผ่านบัญชีลูกค้าด้วย Bot จำนวน 1,000 ครั้ง/นาที                       |
| Artifact         | Authentication Service และ API Gateway                                                      |
| Environment      | ระบบทำงาน Production ตลอดเวลา                                                               |
| Response         | ระบบ Block IP ที่ผิดปกติ, ล็อกบัญชีชั่วคราวหลังผิด 5 ครั้ง และส่งอีเมลแจ้งเตือนเจ้าของบัญชี |
| Response Measure | ตรวจพบและ Block การโจมตีภายใน 10 วินาที, Rate Limit ≤ 5 Login Attempts/IP/นาที              |

คำอธิบาย: ระบบเก็บข้อมูลส่วนตัวและข้อมูลการชำระเงินของลูกค้า ความปลอดภัยจึงเป็นสิ่งสำคัญสูงสุด การโจมตีแบบ Credential Stuffing เป็น Threat ที่พบบ่อยในแอปพลิเคชันประเภทนี้

QA-5: Usability (ความง่ายในการใช้งาน)

| ส่วนประกอบ       | รายละเอียด                                                                        |
|------------------|-----------------------------------------------------------------------------------|
| Source           | ลูกค้าใหม่ที่ไม่เคยใช้แอปสั่งอาหารมาก่อน                                          |
| Stimulus         | ผู้ใช้ใหม่เปิดแอปฯ เป็นครั้งแรกและต้องการสั่งอาหาร                                |
| Artifact         | Customer Mobile Application (UI/UX Flow)                                          |
| Environment      | ผู้ใช้ใช้งานในสภาพปกติ ไม่มีคู่มือ                                                |
| Response         | ผู้ใช้สามารถค้นหาร้าน เลือกเมนู และชำระเงินได้สำเร็จโดยไม่ต้องได้รับความช่วยเหลือ |
| Response Measure | ผู้ใช้ใหม่ 80% สามารถสั่งอาหารได้สำเร็จภายใน 3 นาที โดยไม่เปิด Help Center        |

คำอธิบาย: แอปพลิเคชันที่ใช้งานง่ายจะสูญเสียลูกค้าในทันที การออกแบบ UI/UX ที่ Intuitive และลด Cognitive Load เป็นปัจจัยที่ส่งผลต่อ Conversion Rate โดยตรง

QA-6: Maintainability (ความง่ายในการบำรุงรักษา)

| ส่วนประกอบ       | รายละเอียด                                                                        |
|------------------|-----------------------------------------------------------------------------------|
| Source           | นักพัฒนาในทีม Backend                                                             |
| Stimulus         | ต้องการเพิ่ม Payment Provider ใหม่ (เช่น TrueMoney Wallet)                        |
| Artifact         | Payment Service Module                                                            |
| Environment      | ระบบทำงาน Production ระหว่างการพัฒนา Feature ใหม่                                 |
| Response         | นักพัฒนาสามารถเพิ่ม Payment Provider ใหม่โดยไม่ต้องแก้ไข Core Business Logic เดิม |
| Response Measure | ใช้เวลา Development ≤ 3 วันทำการ และผ่าน Regression Test ก่อน Deploy              |

คำอธิบาย: ตลาด Payment ในไทยมี Provider หลายรายและเปลี่ยนแปลงบ่อย การออกแบบ Payment Module ให้รองรับ Plugin Pattern (Strategy Design Pattern) จะลดเวลาในการ Integrate Provider ใหม่อย่างมีนัยสำคัญ

ส่วนที่ 4: Constraints

Technical Constraints

- ระบบต้องทำงานบน iOS 14+ และ Android 10+ เพื่อรองรับกลุ่มผู้ใช้หลัก
- ต้องใช้ HTTPS/TLS 1.3 สำหรับการสื่อสารทุกประเภทเพื่อความปลอดภัย
- Database ต้องรองรับ ACID Transaction สำหรับการชำระเงินและการสร้าง Order
- ระบบแผนที่ต้องใช้ Google Maps API หรือ Longdo Map (API ที่ได้รับอนุญาต)
- ต้องรองรับ Concurrent Users ขั้นต่ำ 10,000 คนโดยไม่เกิด Degradation

Time Constraints

- ต้องพัฒนา MVP ให้แล้วเสร็จภายใน 16 สัปดาห์ตามกำหนดการของรายวิชา
- ต้องส่งเอกสาร Architectural Drivers ภายใน 1 สัปดาห์นับจากวันมอบหมาย

Budget Constraints

- ใช้ Cloud Infrastructure แบบ Free Tier / Student Credit ในระยะพัฒนา
- ใช้ Open-source Libraries และ Frameworks เพื่อลดค่าใช้จ่าย License
- ค่า Third-party API (เช่น Google Maps, SMS OTP) ต้องอยู่ในงบประมาณที่กำหนด

Legal / Policy Constraints

- ต้องปฏิบัติตาม พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล (PDPA) พ.ศ. 2562 ในการเก็บและใช้ข้อมูลลูกค้า
- ข้อมูลบัตรเครดิตต้องผ่านมาตรฐาน PCI DSS หรือใช้ Payment Gateway ที่ได้รับการรับรองแทน
- ต้องมีนโยบาย Privacy Policy และ Terms of Service ที่ครบถ้วนก่อน Go-live

ส่วนที่ 5: Assumptions

1. ผู้ใช้ทุกคนมีสมาร์ทโฟนที่เชื่อมต่ออินเทอร์เน็ตได้และมีแอปฯ ติดตั้งแล้ว
2. ร้านอาหารพาร์ทเนอร์มีความสามารถรับ Notification และยืนยัน Order ได้ทันที
3. พนักงาน Rider มีสมาร์ทโฟนและ GPS ทำงานได้อย่างถูกต้อง
4. ระบบ Payment Gateway (เช่น Omise, KBank) พร้อมใช้งานและมี Uptime ≥ 99%
5. ข้อมูลตำแหน่งร้านอาหารและขอบเขตพื้นที่ให้บริการถูกต้องและอัปเดตแล้ว
6. กฎหมาย PDPA ที่บังคับใช้อยู่ปัจจุบันไม่มีการเปลี่ยนแปลงอย่างมีนัยสำคัญในช่วงพัฒนา

ส่วนที่ 6: Priority & Trade-offs

Quality Attributes Priority

| อันดับ | Quality Attribute | เหตุผล                                                                                         |
|--------|-------------------|------------------------------------------------------------------------------------------------|
| 1      | Availability      | ระบบต้องทำงานได้ตลอด 24/7 หากล่มแม้ครู่เดียวส่งผลกระทบต่อรายได้ร้านอาหาร Rider และลูกค้าโดยตรง |
| 2      | Security          | ระบบเก็บข้อมูลส่วนตัวและการชำระเงิน หากถูกโจมตีจะสูญเสียความไว้วางใจของผู้ใช้และมีผลทางกฎหมาย  |
| 3      | Performance       | ช่วง Peak Hour ระบบต้องตอบสนองรวดเร็ว ความช้าทำให้ผู้ใช้ยกเลิกการสั่งและเปลี่ยนไปใช้คู่แข่ง    |
| 4      | Scalability       | ธุรกิจ Food Delivery เติบโตเร็ว ระบบต้องรองรับ User ที่เพิ่มขึ้นโดยไม่ต้อง Redesign ใหม่       |
| 5      | Usability         | ผู้ใช้งานไม่กลับมาหากแอปใช้งานยาก ส่งผลต่อ Retention Rate และความสำเร็จของธุรกิจ               |
| 6      | Maintainability   | ระบบต้องพัฒนาต่อได้ง่าย รองรับ Feature ใหม่และ Integration กับพาร์ทเนอร์ได้รวดเร็ว             |

Trade-offs Analysis

Trade-off #1: Performance vs. Consistency

ระบบสั่งอาหารต้องตัดสินใจระหว่างการตอบสนองที่รวดเร็ว (Performance) กับความถูกต้องของข้อมูล (Consistency) โดยเฉพาะในส่วนของ Real-time Tracking และสถานะ Order ถ้าเลือก Strong Consistency ระบบต้องรอให้ทุก Node อัปเดตข้อมูลพร้อมกัน ซึ่งเพิ่ม Latency อย่างมาก แต่ถ้าใช้ Eventual Consistency ระบบตอบสนองเร็วแต่อาจแสดงข้อมูลที่ล้าหลังเล็กน้อย

Decision: เลือกใช้ Eventual Consistency สำหรับข้อมูล Tracking และสถานะที่ไม่ใช่ Critical เช่น ตำแหน่ง Rider บนแผนที่ (ยอมรับ Delay 2-3 วินาที) แต่ยังคง Strong Consistency สำหรับข้อมูลการชำระเงินและสถานะ Order ที่ส่งผลกระทบต่อรายได้ การแบ่ง Consistency Policy แบบ Hybrid นี้ทำให้ระบบสมดุลระหว่างประสบการณ์ผู้ใช้กับความถูกต้องของข้อมูล

Trade-off #2: Availability vs. Security

การเพิ่ม Security Layer มากขึ้น เช่น MFA ทุก Session, Re-authentication ทุก Transaction ทำให้ระบบปลอดภัยแต่เพิ่ม Friction ให้ผู้ใช้ และอาจนำไปสู่การ Drop-off ในกระบวนการสั่งอาหาร ขณะเดียวกัน การลด Security Check เพื่อให้ Checkout ไว ก็เพิ่มความเสี่ยง Fraud

Decision: ออกแบบตาม Risk-based Authentication โดยใช้ MFA เฉพาะเมื่อมี Suspicious Activity (เช่น Login จาก Device ใหม่ หรือ IP ต่างประเทศ) และใช้ Token-based Session ที่ Expire ใน 7 วัน แนวทางนี้ช่วยรักษาประสบการณ์ผู้ใช้ในกรณีปกติ ขณะที่ยังคงมี Security ที่แข็งแกร่งเมื่อจำเป็น

Trade-off #3: Scalability vs. Cost

การออกแบบระบบให้ Scalable อย่างเต็มรูปแบบ (เช่น Microservices ทุก Module, Auto-scaling ทุก Layer) ใช้ต้นทุน Infrastructure สูงมาก แต่การประหยัดต้นทุนด้วยการใช้ Monolith จะทำให้ขยายระบบได้ยากเมื่อผู้ใช้เพิ่ม

Decision: เลือกใช้ Modular Monolith ในระยะแรก (MVP) โดยแยก Module ให้ชัดเจน เพื่อให้ง่ายต่อการ Migrate เป็น Microservices ในอนาคตเมื่อ Traffic เติบโตถึงจุดที่คุ้มค่า กลยุทธ์นี้ช่วยประหยัด Cost ในช่วงเริ่มต้น ขณะที่ยังคงรักษา Architectural Path สู่ Scalability ในอนาคต